

8. Postman Collection Structure:

In this section, we design a **well-organized Postman collection**, which represents the complete API test suite in a structured folder format. This helps in scalability, readability, and easy maintenance.

1. Authentication

This folder handles all login-related APIs.

◆ Login User

- Endpoint is used to authenticate a user.
- We send username and password in request body.
- On successful login, API returns a **token**.
- This token is stored in an environment variable (example: `authToken`).

🔑 Purpose:

- To simulate real-world secured APIs.
 - Token is reused in Products, Cart, and Users APIs.
-

2. Products

This is the most important module because it deals with e-commerce items.

◆ Get All Products

- Fetches complete product list.
- Validates status code (200).
- Checks response structure (array format).

◆ Get Single Product

- Fetch product using product ID.
- Validate specific fields like title, price, category.

◆ Add New Product (POST)

- Sends product data (title, price, description).
- Validates:
 - Status code (200/201)
 - Response contains generated ID

◆ Update Product (PUT)

- Fully replaces product data.
- All fields must be sent.

◆ Update Product (PATCH)

- Partial update (only selected fields like price or title).

◆ Delete Product

- Deletes product by ID.
- Validate success message or status code.

☞ Purpose:

- Covers full CRUD operations (Create, Read, Update, Delete).
-

3. Categories

This module is used to test product grouping logic.

◆ Get All Categories

- Returns all available product categories.

◆ Get Products in Category

- Fetch products belonging to a specific category (e.g. electronics).
- Validates filtered response data.

☞ Purpose:

- Ensures filtering and categorization logic is working properly.
-

4. Cart

This simulates shopping cart functionality.

◆ Get All Carts

- Retrieves all cart data from system.

◆ Get Single Cart

- Fetch cart by ID and validate items inside it.

◆ Add to Cart (POST)

- Adds products into cart.
- Includes product ID and quantity.

◆ Update Cart (PUT)

- Fully updates cart content.

◆ Update Cart (PATCH)

- Partially updates cart (e.g. quantity change).

◆ Delete Cart

- Removes cart from system.

☞ Purpose:

- Simulates real e-commerce checkout behavior.
-

5. Users

Handles user-related API testing.

◆ Get All Users

- Fetch all registered users.

◆ Get Single User

- Fetch specific user using ID.
- Validate fields like name, email, phone.

☞ Purpose:

- Ensures user management APIs are working correctly.